

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a2b031f1315692dca.jsonl`  
- SHA-256: `5aa34435256252b0f1a73e640745f9ecef4c7ce80578b113d923e70fbcdbdf7d`  
- Source modified: `2026-07-06T19:18:19+00:00`  
- Imported at: `2026-07-08T16:00:11+00:00`  
- project: `wf_e1b943dd-dac`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T19:16:00.130Z)

Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR (branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F design doc for a four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content safety / Tier 4 deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up issues F1-F7.
No code changes. Read the ADR:
git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the codebase and the other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem, Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate should_ingest_source, delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler: watched gate -> dedup -> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track C (delivery_attempts) just merged.

Ground rules:

- READ actual files, cite ADR section / file:line. You MAY run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does FilterConfig ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies, not merge-blockers.
 - Severity: high (a claim that is false, or a design gap that would break an F-phase / cause data loss), medium (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example, minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for implementation.

DIMENSION: design & interface soundness / completeness.
Investigate:
- Is PolicyContext (content_item, source, policy) SUFFICIENT for the tiers it defines? SEED to verify/refute:
Tier 3 (light I/O) and Tier 4 (download/ffprobe) need a client/message handle to fetch anything, but
PolicyContext has none ? so the I/O tiers can't do I/O as specified. Also Tier 2/3 do caption/keyword matching
which needs content_item.text ? but ADR 0011 says ContentItem.text is NULL until a later track populates it
(check models.py + whether anything writes text). Unstated dependency?
- The pipeline is AND-only (short-circuit on first failure). Any expressiveness gap the ADR should acknowledge?
- StageResult recorded via delivery_attempts.last_error "for failed items" ? but rejected items short-circuit
BEFORE delivery, so is there even a delivery_attempts row to write to? Category error?
- Phasing F1-F7 dependencies sane? Anything a phase needs that the ADR doesn't provide in the contract?
Return findings.

2. assistant (2026-07-06T19:16:04.822Z)

I'll review this ADR. Let me start by reading it and the relevant code.

3. user (2026-07-06T19:16:06.822Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config\_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state
--- ---	
`FilterConfig` / `Policy`	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.
`FilterEngine`	Evaluates all rules; returns `FilterResult(passed, reasons)`.
`policies` table	One row per account (is_default=1). `config_json` is the `FilterConfig` blob.
Security / topic / deep-inspect	Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan.

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express

topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
...
????????????????????????????????????????????????????????????
?          Policy Pipeline                                     ?
?                                                         ?
? Tier 1 ? Metadata gates  (always runs, zero I/O)          ?
?   duration, resolution, size, codec, mime, GIF/round      ?
?   ? extends today's FilterEngine                          ?
?                                                         ?
? Tier 2 ? Topic preferences (opt-in, metadata-only)        ?
?   source allowlist, keyword match, channel topic tags     ?
?   ? user-expressed interests filter the stream           ?
?                                                         ?
? Tier 3 ? Content safety  (opt-in, metadata + light I/O)   ?
?   URL/domain reputation, file metadata sanity,           ?
?   caption text scanning                                  ?
?   ? cheap checks; no download                            ?
?                                                         ?
? Tier 4 ? Deep inspection (opt-in, heavy I/O)              ?
?   ffprompt, perceptual hash, downloaded content scan     ?
?   ? gated behind explicit user opt-in + budget           ?
????????????????????????????????????????????????????????????
...
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```
```python
@dataclass
class PolicyContext:
 """Immutable snapshot of what the stage can inspect."""
 content_item: ContentItem # from ADR 0011
 source: Source # the source it came from
 policy: Policy # the owning policy config

@dataclass
class StageResult:
 passed: bool
 reason: str # human-readable explanation
 stage_name: str # e.g. "duration-gate", "url-reputation"
 metadata: dict[str, object] = field(default_factory=dict) # for explainability UI

class PolicyStage(Protocol):
 """A single gate in the pipeline. Stateless; may be async for I/O stages."""
 async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
...
```
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config\_json` under a `topics` key:

```
```json
{
 "min_duration_s": 5,
 "topics": {
 "include": ["AI", "Philosophy", "Soccer"],
 "exclude": ["Politics", "Crypto"],
 "mode": "any" // "any" = match any include topic; "all" = match all
 }
}
```
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider_source_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content\_fingerprint` across accounts (already handled by `content\_items.dedup\_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked\_domain": "sketchy-site.com"}`).

Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated** behind explicit opt-in because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep\_inspect: true` in `config\_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

Policy evaluation flow (post-dedup, pre-delivery)

```
```python
async def evaluate_pipeline(
 ctx: PolicyContext,
 stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
 results: list[StageResult] = []
 for stage in stages:
 result = await stage.evaluate(ctx)
 results.append(result)
 if not result.passed:
 return False, results # short-circuit
 return True, results
```
```

...

The caller (telethon_client or future Bot API handler) records the results via `delivery\_attempts.last\_error` (for failed items) or a new `policy\_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

****No schema migration**** is required for this design. The `policies` table already holds a `config\_json` JSON blob. The pipeline is a code concept:

- `policies.config\_json` gains optional `topics`, `allowlist`, `security\_checks`, `deep\_inspect` keys ? all backwards-compatible with the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy\_decisions` table (deferred) would store the explanation trace for debugging: `(content\_item\_id, policy\_id, stage\_name, passed, reason, metadata\_json, evaluated\_at)`.

Implementation phasing (future issues)

| Phase | Scope | Dependencies |
|---------------|-------------------------|--|
| --- | --- | --- |
| **F1** | Policy engine prototype | `PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler None |
| **F2** | Topic preferences | `topics` config parsing, source title/caption keyword matching F1 |
| **F3** | Source allowlist | `allowlist` config, skip non-allowlisted sources F1 |
| **F4** | URL/domain reputation | Domain blocklist, caption link extraction F1 |
| **F5** | Deep inspection | ffprobe/perceptual hash, download budget enforcement F1, ADR 0003 waiver |
| **F6** | Policy decisions table | Audit trail storage, explainability UI F1 |
| **F7** | Explainability endpoint | `/why <content_item_id>` command showing the explanation trace F6 |

Each phase is a separate, mergeable PR ? none are bundled here. This ADR only defines the ****design contract**** the phases code against.

Consequences

****Positive****

- Unifies topic, security, and quality gates into a single explainable framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without additional DB lookups at decision time.
- Backwards-compatible: `policies.config\_json` already exists; adding optional keys requires no migration.
- Each implementation phase is independently shippable behind feature flags.

****Negative****

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same interface, which may be awkward for deeply different checks (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not semantic. A future ML stage would replace it, but it's good enough for a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a budget to avoid runaway costs.

Alternatives considered

- ****Do nothing; keep flat FilterConfig.**** Rejected ? the umbrella exists to dissolve the conflation of quality, topic, and security into a single boolean. A pipeline is the natural generalization.
- ****Inline each stage in the handler**** (no protocol). Rejected ? makes it impossible to test stages independently, reorder them, or add new ones without touching the hot path.
- ****Use a rules engine / DSL.**** Rejected ? over-engineered for <100 users. A Python list of callables is simpler, testable, and introspectable.
- ****Bundle all phases into a single PR.**** Rejected ? scope discipline. Each phase is independently shippable; this ADR defines the shared interface they code against.

Open questions for the decision owner

1. ****`StageResult.metadata` shape**** ? dict or typed per-stage? Recommend: dict for Phase 1 (flexible), typed in Phase 2 (once the shape stabilizes).
2. ****Policy decisions table**** ? store every decision, or only failures? Recommend: every decision (explainability), with a TTL-based cleanup for old traces.
3. ****Topic matching**** ? substring only, or regex/glob? Recommend: substring (case-insensitive) for v1; natural-language matching in a follow-up.
4. ****Deep inspection budget**** ? per-item byte limit or per-account bandwidth cap? Recommend: per-item limit (10 MB default) with a per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier 4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:16:08.782Z)

Let me read the relevant code and other ADRs to verify the ADR's claims.

5. user (2026-07-06T19:16:10.936Z)

```
"""Pydantic data models for users, channels, videos, and filter config.
```

```
These models are the in-memory representation used by the repository layer
(`db.py`) and the filter engine (`filters/`). They are deliberately
decoupled from Telethon's own types so the filter engine stays pure and
unit-testable without a live Telegram connection.
"""
```

```
from __future__ import annotations
```

```
from datetime import UTC, datetime
from enum import Enum
```

```
from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
```

```
def _utcnow() -> datetime:
    """Timezone-aware UTC 'now', used as a default factory."""
    return datetime.now(UTC)
```

```

class DeliveryMode(str, Enum):
    """How a matched video is delivered to the user.

    - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
    - ``LINK``: send a text message with a time link + metadata summary.
    - ``BOTH``: forward the video AND send the link message.
    """

    FORWARD = "forward"
    LINK = "link"
    BOTH = "both"

class Account(BaseModel):
    """A product identity ? the owner of config, channels, routes, and content.

    Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
    is the bridge used by the Bot API front-door (``from.id``) and by the
    legacy userbot path; it is nullable so future providers that don't have a
    Telegram id (e.g. WhatsApp) can still own an ``Account``.
    """

    model_config = ConfigDict(frozen=True)

    id: int = Field(description="Stable surrogate PK; the owner key for all downstream tables")
    telegram_id: int | None = Field(
        default=None,
        description="Telegram user id (bot from.id OR userbot account id). UNIQUE when set.",
    )
    provider: str = Field(default="telegram", description="Provider namespace; future: 'whatsapp'")
    display_name: str | None = None
    created_at: datetime = Field(default_factory=utcnow)

class TelegramSession(BaseModel):
    """An optional MTPROTO (Telethon) session linked to an :class:`Account`.

    A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
    that is what makes them structurally invisible to ``manager.run``
    (ADR 0010). ``session_path`` is data-driven so the file-location
    convention is explicit rather than derived from the PK at runtime.
    """

    model_config = ConfigDict(frozen=True)

    account_id: int = Field(description="FK ? accounts.id; PK (one session per account in v1)")
    phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
    session_path: str = Field(description="File-backed Telethon session path (user_<id>.session)")
    enabled: bool = Field(
        default=True,
        description="If False, the manager skips this session (not the account)",
    )
    created_at: datetime = Field(default_factory=utcnow)
    last_login_at: datetime | None = None

class User(BaseModel):
    """Deprecated: legacy v1 model conflating identity + MTPROTO session.

    Retained as a backwards-compatible view over the new ``accounts`` +
    ``telegram_sessions`` tables (ADR 0010). New code should use
    :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
    maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers

```

```
and tests keep working without a flag day.
"""
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy rows)")
phone: str | None = Field(default=None, description="E.164 phone; used only for bootstrap")
enabled: bool = Field(default=True, description="If False, the manager skips this session")
created_at: datetime = Field(default_factory=_utcnow)
```

```
class Channel(BaseModel):
```

```
    """A channel/group a user is a member of. Populated lazily as messages arrive.
```

```
    Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
    code should use :class:`Source` directly; the ``db`` layer maps
    ``Channel`` ? ``Source`` so existing callers and tests keep working.
    """
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
user_id: int
channel_id: int = Field(description="Telegram channel/chat id")
title: str | None = None
seen_first_at: datetime = Field(default_factory=_utcnow)
```

--- provider-neutral domain models (ADR 0011, Track B) -----

```
#
```

These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig`` shapes so the schema no longer leaks Telegram-specific ids into every future provider's code path. ``provider`` columns + provider-prefixed ``dedup\_key`` make cross-provider collisions structurally impossible. See ADR 0011 for the full rationale and the two decisions (Option 1A: tiered ``dedup\_key``; Option 2A: ``policies`` generalizes ``filter\_config``).

```
class Source(BaseModel):
```

```
    """A configured place we ingest content from (ADR 0011).
```

```
    Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
    providers add their own source kinds). ``provider_source_id`` is the
    provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
    """
```

```
model_config = ConfigDict(frozen=True)
```

```
id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
account_id: int = Field(description="FK ? accounts.id (owner of the source)")
provider: str = Field(default="telegram", description="Provider namespace")
provider_source_id: str = Field(
    description="Provider-native source id (Telegram channel_id, ?)"
)
title: str | None = None
kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
watched: bool = Field(
    default=False,
    description="ADR 0009 selective scanning: ingest from this source iff True",
)
```



```

seen_first_at: datetime = Field(default_factory=_utcnow)
last_seen_at: datetime | None = None

class ContentItem(BaseModel):
    """A normalized content record discovered from any :class:`Source` (ADR 0011).

    Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
    (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
    NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
    ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
    wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    source_id: int = Field(description="FK ? sources.id")
    provider: str = Field(default="telegram", description="Provider namespace")
    provider_message_id: str = Field(
        description="Provider-native message id (Telegram message_id, ?)"
    )
    content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
    text: str | None = Field(default=None, description="Caption / post text (nullable in Phase 1)")
    link_url: str | None = Field(default=None, description="For link/article posts (Phase 1: NULL)")
    media_ref: str | None = Field(default=None, description="Provider media reference")
    document_id: str | None = Field(
        default=None,
        description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
    )
    duration_s: int | None = None
    width: int | None = None
    height: int | None = None
    size_bytes: int | None = None
    mime_type: str | None = None
    dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique per account")
    matched: bool = False
    seen_at: datetime = Field(default_factory=_utcnow)

class Policy(BaseModel):
    """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).

    Generalizes the legacy per-account ``filter_config`` row so one account can
    own multiple policies (enables ADR 0009's per-route filter overrides) and so
    future enrichment/security stages grow into the same ``config_json`` column.
    Exactly one policy per account carries ``is_default=True``; the legacy
    ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    name: str = Field(description="Policy name; 'default' for the account default policy")
    config_json: str = Field(
        description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
    )
    is_default: bool = Field(
        default=False, description="Exactly one per account (UNIQUE partial index)"
    )

```

```
created_at: datetime = Field(default_factory=_utcnow)
updated_at: datetime = Field(default_factory=_utcnow)
```

--- Track C: destinations, routing, delivery state (ADR 0011) -----

#

These tables were created empty by Track B (_SCHEMA); Track C populates destinations (backfilled from filter_config_deprecated.config_json), writes delivery_attempts (replacing the legacy videos.forwarded boolean), and defines the models for per-destination delivery state. The source/destination invariant (a channel is either source or destination, never both) is enforced at the app level by future selective-scanning logic (Track E, #30).

```
class Destination(BaseModel):
    """A delivery target ? where matched content is sent (ADR 0011, Track C).

    Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
    can have multiple named output channels. Backfilled from the existing
    per-account ``delivery_target``; the default destination is the target used
    when no explicit routing rule matches.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    provider: str = Field(default="telegram", description="Provider namespace")
    target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
    kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
    title: str | None = None
    is_output: bool = Field(
        default=False,
        description="ADR 0009: distinguishes output channels from ingest sources",
    )
    created_at: datetime = Field(default_factory=_utcnow)


class RoutingRule(BaseModel):
    """A rule that maps a source through a policy to a destination (ADR 0009).

    Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
    ``policy_id`` is NULL, the account's default policy is used. When multiple
    rules match, the highest ``priority`` wins.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    account_id: int = Field(description="FK ? accounts.id")
    source_id: int = Field(description="FK ? sources.id")
    policy_id: int | None = Field(
        default=None,
        description="FK ? policies.id; NULL means use account default policy",
    )
    destination_id: int = Field(description="FK ? destinations.id")
    enabled: bool = Field(default=True)
    priority: int = Field(default=0, description="Higher wins when multiple rules match")
    created_at: datetime = Field(default_factory=_utcnow)
```

```

class DeliveryStatus(str, Enum):
    """Per-destination delivery state, replacing the legacy binary ``forwarded``."""

    PENDING = "pending"
    DELIVERED = "delivered"
    FAILED = "failed"
    RETRYING = "retrying"


class DeliveryAttempt(BaseModel):
    """Per-destination delivery record ? idempotent via UNIQUE index.

    Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
    ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
    incremented and ``status``/``last_error`` are updated.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    content_item_id: int = Field(description="FK ? content_items.id")
    destination_id: int = Field(description="FK ? destinations.id")
    route_id: int | None = Field(default=None, description="FK ? routing_rules.id (nullable)")
    status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
    attempt_count: int = Field(default=0, description="Incremented on each retry")
    last_error: str | None = None
    created_at: datetime = Field(default_factory=utcnow)
    delivered_at: datetime | None = None
    next_retry_at: datetime | None = None


class FilterConfig(BaseModel):
    """Per-user filter configuration.

    v1 rules: duration bounds, minimum height (resolution), and max file size.
    Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
    new field later is backwards-compatible.
    """

    # Duration bounds in seconds. None means unbounded on that side.
    min_duration_s: int | None = Field(default=5, ge=0)
    max_duration_s: int | None = Field(default=7200, ge=0)

    # Resolution: a video passes if its height >= target_height.
    # None disables the resolution rule.
    target_height: int | None = Field(default=1080, ge=1)

    # Max file size in mebibytes. None disables the size rule.
    max_size_mb: int | None = Field(default=2048, ge=1)

    # Content-type exclusions. GIFs and round videos are excluded by default
    # (their duration/dimensions are usually incidental to real video filters);
    # silent videos are off by default since some legit clips have no audio.
    # Existing users' stored configs are unaffected ? the DB JSON blob overrides
    # these defaults, so this only changes the behaviour for fresh installs.
    exclude_gif: bool = Field(
        default=True,
        description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
    )
    exclude_round: bool = Field(
        default=True,
        description="If True, exclude circular video messages (round_message=True)",
    )
    exclude_nosound: bool = Field(
        default=False,

```

```

        description="If True, exclude videos without an audio track (nosound=True)",
    )

# How matched videos are delivered: forward the video, send a link, or both.
# Stored in the same JSON blob, so adding this field is backwards-compatible
# (existing rows without it default to FORWARD).
delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)

# Where matched videos / link messages are sent. Accepts:
# - "me" (default): Saved Messages ? exact v1.1 behaviour.
# - "@username": a public channel the account is a member of.
# - "-100..." or a bare int-as-string: a numeric Telegram chat id
#   (private channels without a username).
# The string is passed verbatim to Telethon's forward_messages / send_message.
# See ADR 0005 for rationale and access requirements.
delivery_target: str = Field(
    default="me",
    min_length=1,
    description="Telethon entity to deliver to: 'me', '@username', or a chat id",
)

@field_validator("delivery_target", mode="before")
@classmethod
def _coerce_delivery_target(cls, v: object) -> str:
    """Accept int chat ids (e.g. -1001234567890) and coerce to str.

    Telethon accepts both str and int as an entity, but storing as str keeps
    the JSON blob uniform and lets '@username' / 'me' / numeric ids share
    one field type.
    """
    if v is None:
        return "me"
    if isinstance(v, int):
        return str(v)
    return str(v)

@model_validator(mode="after")
def _check_duration_bounds(self) -> FilterConfig:
    """Ensure min_duration_s <= max_duration_s when both are set."""
    if (
        self.min_duration_s is not None
        and self.max_duration_s is not None
        and self.min_duration_s > self.max_duration_s
    ):
        raise ValueError(
            f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
            f"({self.max_duration_s})"
        )
    return self

class VideoRecord(BaseModel):
    """A seen video message + its metadata. Drives dedup and future /search.

    Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
    New code should use :class:`ContentItem`; the ``db`` layer maps
    ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
    """

    model_config = ConfigDict(frozen=True)

    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
    user_id: int
    channel_id: int = Field(description="Origin Telegram channel id")
    message_id: int = Field(description="Telegram message id within the channel")

```

```

document_id: int | None = Field(
    default=None,
    description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
)

# Metadata extracted from the Telethon message (may be None if absent).
duration_s: int | None = None
width: int | None = None
height: int | None = None
size_bytes: int | None = None
mime_type: str | None = None

matched: bool = False
forwarded: bool = False
seen_at: datetime = Field(default_factory=utcnow)

@property
def dedup_key(self) -> tuple[int, int, int]:
    """The natural key used for deduplication in the ``videos`` table."""
    return (self.user_id, self.channel_id, self.message_id)

@property
def content_fingerprint(self) -> tuple[int, int, int, int] | None:
    """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height, width).

    Returns None if any field is missing (cannot fingerprint).
    """
    if None in (self.size_bytes, self.duration_s, self.height, self.width):
        return None
    return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]

```

Default filter config shared by the bootstrap onboarding step.

```
DEFAULT_FILTER_CONFIG = FilterConfig()
```

6. user (2026-07-06T19:16:12.337Z)

ADR 0011: Provider-neutral domain schema (Track B)

- **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram\_sessions`, merged in #33)
- **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly #30 (Track E)

Context

ADR 0010 split `users` into `accounts` (product identity) + `telegram\_sessions` (MTPROTO credential). That unblocked the *identity* dimension. This ADR tackles the **content dimension**: how to represent sources, content items, policies, destinations, routes, and delivery attempts in a provider-neutral way, so that the system can ingest from Telegram today and from WhatsApp (or any future provider) later without another schema rewrite.

What we have today (post-Track-A)

| Table | Meaning | Provider-coupling |
|---------------------|-------------------------------------|--------------------------------------|
| --- --- --- | | |
| `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
| `telegram_sessions` | Optional MTPROTO session (ADR 0010) | Telegram-specific by design |

| `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel\_id` is a Telegram id) |
| `videos` | A seen Telegram video message + metadata | **Telegram-specific** (`message\_id`,
`document\_id` are Telegram ids) |
| `filter\_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape, but
scoped per-account not per-source |

The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:

- Tier 0: `(user\_id, channel\_id, message\_id)` ? same Telegram message.
- Tier 1: `(user\_id, document\_id)` ? same Telegram file forwarded across channels.
- Tier 2: `(user\_id, size\_bytes, duration\_s, height, width)` ? re-upload fingerprint.

What we need

The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`, `Destination`, `Route` (`routing\_rules`), and `DeliveryAttempt`. This ADR defines the schema for each, the **two open design questions** that gate the schema, and the migration path from the current tables.

The two decisions I need from you

Question 1: `ContentItem.dedup\_key` ? how does provider-neutral dedup relate to ADR 0006?

The current 3-tier dedup is Telegram-specific (`document\_id` is a Telegram concept). A provider-neutral `ContentItem` needs a dedup story that works across providers. Three options:

Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers

Keep ADR 0006's three tiers but generalize them behind a single
`dedup\_key` column on `content\_items`:

```
...  
content_items.dedup_key TEXT NOT NULL  
...
```

The *value* of `dedup\_key` is computed by an explicit decision tree at ingestion time, provider by provider. The tree is ordered strongest-key first and each branch is **guarded** so partial metadata can never collapse to a degenerate key (e.g. `tg:fp:0:0:0:0`):

- **Telegram**:
 1. If `document\_id` is present ? `dedup\_key = "tg:doc:{document\_id}"`.
 2. Else if **all four** fingerprint fields (`size\_bytes`, `duration\_s`, `height`, `width`) are present ? `dedup\_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
 3. Else ? `dedup\_key = "tg:msg:{source\_id}:{provider\_message\_id}"`
(Tier 0; the source/channel id is required because Telegram
`message\_id` is only unique within a channel, not across sources).
- **Future WhatsApp**: `dedup\_key = "wa:msg:{message\_id}"` (WhatsApp message
ids are globally unique per account) or `"wa:sha256:{blob\_hash}"` if WA
exposes a content hash.

The guard in step 2 (all four fields required) is what prevents the partial-metadata collision the first draft of this ADR allowed. The pure
`compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
size_bytes, duration_s, height, width)` function that implements this tree
gets its own unit tests in the implementation PR, including cases for
partial/null metadata.

A **partial unique index** on `(account\_id, dedup\_key) WHERE dedup\_key IS NOT NULL`
replaces the current Tier-1 index and enforces per-account dedup at the DB
level, regardless of provider.

Why recommended:

- Zero new bandwidth ? all keys are computed from metadata already extracted (honors ADR 0003's no-download principle).
- The provider prefix (`tg:` / `wa:`) makes cross-provider collisions structurally impossible.
- Existing `videos` rows migrate cleanly: `dedup\_key` is computed from their existing `document\_id` / metadata columns during the migration.
- Supersedes ADR 0006 cleanly: the three tiers become *one* column with provider-specific derivation rules, documented in the migration.

****Costs:****

- The migration must compute `dedup\_key` for every existing `videos` row (cheap ? pure SQL string building, no I/O).
- The `dedup\_key` derivation logic lives in code (a pure function per provider), not in the schema. This is a feature, but it means the derivation rules need their own unit tests.

Option 1B: content hash (SHA-256 of downloaded bytes)

`dedup\_key` = sha256(downloaded_content)`. Cryptographically strong, provider-agnostic, catches re-encodes that 1A misses.

****Rejected**** ? violates ADR 0003 (metadata-first, no download in the hot path). Downloading every incoming video to hash it would multiply bandwidth and latency by orders of magnitude. Could be added later as an optional deep-inspect stage (deferred to Track F per #31).

Option 1C: provider-assigned only (drop Tier 2)

`dedup\_key` = "tg:doc:{document_id}"` always; no metadata fingerprint fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new `document.id` ? a real and common case (someone re-encodes and re-uploads the same video to a different channel).

****Recommendation: 1A.**** It preserves the dedup quality of ADR 0006, generalizes cleanly, and honors the no-download constraint.

Question 2: `Policy` ? existing `filter\_config` ? rename, or wrap?

The codebase already has `filter\_config.config\_json` (one row per account, holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The umbrella introduces a `Policy` concept. They overlap. Two options:

Option 2A (recommended): `Policy` *generalizes* `filter\_config` (rename + extend)

`policies` is a new table; `filter\_config` is migrated into it. One account can own multiple policies (enables per-route filter overrides from ADR 0009), with a `DEFAULT` policy per account for the no-rules-configured case.

...

```
policies (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  account_id  INTEGER NOT NULL REFERENCES accounts(id),
  name        TEXT NOT NULL,                -- e.g. "default", "reggae-route"
  config_json TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)
  is_default  INTEGER NOT NULL DEFAULT 0,   -- one per account (UNIQUE partial idx)
  created_at  TEXT NOT NULL,
  updated_at  TEXT NOT NULL,
  UNIQUE (account_id, name)
);
...
```

****Phase 1**** (this track): `config\_json` holds exactly today's `FilterConfig` JSON ? the rename is purely structural. ****Phase 2**** (Track F, deferred):

`config\_json` grows to include topic preferences and security rules, but that's behind the same column so no further migration.

Migration: for each `filter\_config` row, insert a `policies` row with `name='default'`, `is\_default=1`, `config\_json=<the existing blob>`. Drop `filter\_config` after the migration (keep `filter\_config\_deprecated` for one release, mirroring ADR 0010's safety-net pattern).

****Why recommended:****

- `Policy` becomes the single home for "what to filter/enrich" ? no second overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
- Enables ADR 0009's per-route filter overrides (`routing\_rules.policy\_id`) without a second migration later.
- The existing `db.load\_filter\_config(account\_id)` / `save\_filter\_config` become thin shims over `policies WHERE account\_id=? AND is\_default=1`, mirroring the Track-A `User` shim pattern ? backwards-compatible.

****Costs:****

- One more table migration. `filter\_config` ? `policies` is a rename+extend.
- The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape); `Policy` is the *container* (name, owner, default flag) around it. Two models, not one ? but they compose cleanly.

Option 2B: keep `filter\_config` as-is; `Policy` is a wrapping layer

Don't touch `filter\_config`. `policies` references a `filter\_config` row plus adds enrichment/security stages as separate columns.

****Rejected**** ? creates two overlapping concepts (`filter\_config` for the v1 rules, `policies` for "everything else"), which is exactly the conflation the umbrella exists to dissolve. We'd be back here in a year merging them.

****Recommendation: 2A.**** One concept, one table, phased growth via the same JSON column.

Proposed schema (assuming 1A + 2A)

`sources` (generalizes `channels`)

A configured place we ingest from. Maps 1:1 to today's per-account `channels` rows for Telegram; future providers add their own source kinds.

...

```
sources (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  account_id        INTEGER NOT NULL REFERENCES accounts(id),
  provider          TEXT NOT NULL DEFAULT 'telegram',
  provider_source_id TEXT NOT NULL,           -- Telegram channel_id; future: WA group id
  title             TEXT,
  kind              TEXT NOT NULL DEFAULT 'channel', -- 'channel' | 'group' | 'feed'
  watched           INTEGER NOT NULL DEFAULT 0,    -- ADR 0009 selective scanning
  seen_first_at     TEXT NOT NULL,
  last_seen_at      TEXT,
  UNIQUE (account_id, provider, provider_source_id)
);
...
```

`channels.user\_id` (now ? `accounts.id`) maps to `sources.account\_id`;
`channels.channel\_id` maps to `sources.provider\_source\_id`. The `watched` flag (ADR 0009, deferred to Track E) is included here so Track E doesn't need another migration.

`content\_items` (generalizes `videos`)

A normalized item discovered from any source. Phase 1 stores only videos (matches today's `videos`); the columns are provider-neutral but the Phase-1 migration only populates video fields.

...

```
content_items (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  account_id        INTEGER NOT NULL REFERENCES accounts(id),
  source_id         INTEGER NOT NULL REFERENCES sources(id),
  provider          TEXT NOT NULL DEFAULT 'telegram',
  provider_message_id TEXT NOT NULL,          -- Telegram message_id; future: WA message id
  content_type      TEXT NOT NULL DEFAULT 'video', -- 'video' | 'link' | 'article' | ...
  text              TEXT,                      -- caption / post text (nullable)
  link_url          TEXT,                      -- for link/article posts
  media_ref         TEXT,                      -- provider media reference (file path, blob id)
  document_id       TEXT,                      -- Telegram document.id (kept for TG-specific
lookups)
  duration_s        INTEGER,
  width             INTEGER,
  height            INTEGER,
  size_bytes        INTEGER,
  mime_type         TEXT,
  dedup_key         TEXT NOT NULL,            -- ADR 0011 Question 1 (Option 1A)
  matched           INTEGER NOT NULL DEFAULT 0,
  seen_at           TEXT NOT NULL,
  -- Tier-0 equivalent: provider_message_id is only unique within a source
  -- (a Telegram message_id is per-channel), so source_id is required to
  -- avoid false collisions across sources for the same account.
  UNIQUE (account_id, source_id, provider_message_id),
  UNIQUE (account_id, dedup_key)              -- Tier-1/2 equivalent (Option 1A)
);
...
```

****Note on `forwarded`:**** dropped from `content\_items`. Per-destination delivery state moves to `delivery\_attempts` (Track C, #28). For Phase 1 (before Track C lands), `content\_items.matched` alone is sufficient ? delivery still goes to the single `delivery\_target`, and the boolean `forwarded` is retained temporarily on a **deprecated** `videos`-compat view so the v1 delivery path keeps working. Track C removes it.

`policies` (generalizes `filter\_config`) ? Option 2A

As specified in Question 2 above.

`destinations` (new ? Track C will populate)

Defined here so the schema is complete, but ****Track C (#28) implements the delivery layer****. Phase-1 presence is the table only (no delivery logic).

...

```
destinations (
  id                INTEGER PRIMARY KEY AUTOINCREMENT,
  account_id        INTEGER NOT NULL REFERENCES accounts(id),
  provider          TEXT NOT NULL DEFAULT 'telegram',
  target            TEXT NOT NULL,            -- 'me', '@username', chat id (ADR 0005)
  kind              TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'
  title             TEXT,
  is_output         INTEGER NOT NULL DEFAULT 0, -- ADR 0009: distinguishes output channels
  created_at        TEXT NOT NULL,
  UNIQUE (account_id, provider, target)
);
...
```

`routing\_rules` (new ? Track C will populate)

Per ADR 0009 (canonical name locked in issue #25).

...

```
routing_rules (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  account_id  INTEGER NOT NULL REFERENCES accounts(id),  
  source_id   INTEGER NOT NULL REFERENCES sources(id),  
  policy_id   INTEGER,                -- nullable ? use account's default policy  
  destination_id INTEGER NOT NULL REFERENCES destinations(id),  
  enabled     INTEGER NOT NULL DEFAULT 1,  
  priority    INTEGER NOT NULL DEFAULT 0,  
  created_at  TEXT NOT NULL,  
  UNIQUE (account_id, source_id, destination_id)  
);  
...
```

`delivery\_attempts` (new ? Track C will populate)

Per #28's idempotency requirement (unique index on
`(content\_item\_id, destination\_id)`).

...

```
delivery_attempts (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  content_item_id INTEGER NOT NULL REFERENCES content_items(id),  
  destination_id INTEGER NOT NULL REFERENCES destinations(id),  
  route_id    INTEGER REFERENCES routing_rules(id),  
  status      TEXT NOT NULL DEFAULT 'pending', --  
'pending' || 'delivered' || 'failed' || 'retrying'  
  attempt_count INTEGER NOT NULL DEFAULT 0,  
  last_error   TEXT,  
  created_at   TEXT NOT NULL,  
  delivered_at TEXT,  
  next_retry_at TEXT,  
  UNIQUE (content_item_id, destination_id)      -- ADR #28 idempotency  
);  
...
```

Migration strategy (integer-preserving, phased)

This track lands **``sources``, ``content\_items``, and ``policies``** only.
``destinations``, ``routing\_rules``, ``delivery\_attempts`` are defined in the
schema (so the ADR is complete) but their tables are created empty and
populated by Track C. This keeps Track B focused and reviewable.

Migration ``\_migration\_003\_domain\_schema`` (idempotent)

1. **``Create``** **``sources``, ``content\_items``, ``policies``, ``destinations``,
``routing\_rules``, ``delivery\_attempts``** (all **``IF NOT EXISTS``**).
2. **``Backfill``** **``sources``** from **``channels``**:
 ``provider='telegram', ``provider_source_id=channel_id``, ``watched=1``
 (ADR 0009 backfill: already-seen channels stay watched).
3. **``Backfill``** **``content\_items``** from **``videos``**, computing **``dedup\_key``** via the
 explicit decision tree from Option 1A (every branch reachable, no
 ``COALESCE(...,0)`` collapsing partial metadata to a degenerate key):
 - If **``document\_id``** IS NOT NULL:
 ``dedup\_key = 'tg:doc:' || document\_id``.
 - Else if **``size\_bytes``** IS NOT NULL AND **``duration\_s``** IS NOT NULL AND **``height``** IS NOT NULL AND **``width``**
 IS NOT NULL:
 ``dedup\_key = 'tg:fp:' || size\_bytes || ':' || duration\_s || ':' || height || ':' || width``.
 - Else (Tier-0 fallback; partial or missing media metadata):
 ``dedup\_key = 'tg:msg:' || source\_id || ':' || provider\_message\_id``,

where ``source_id`` is the joined ``sources.id`` for the row's legacy ``channel_id`` and ``provider_message_id`` is the legacy ``message_id``.

The implementation PR must include migration tests that seed rows hitting each branch (including all-fingerprint-fields-null) and assert no two rows collapse to the same ``dedup_key``.

4. **Backfill** ``policies`` from ``filter_config``: one row per account, ``name`='default'`, ``is_default`=1`, ``config_json`` copied verbatim.
5. **Rename** (don't drop) ``channels`` ? ``channels_deprecated``, ``videos`` ? ``videos_deprecated``, ``filter_config`` ? ``filter_config_deprecated``. Kept for one release as a rollback safety net (ADR 0010 pattern).

The legacy ``videos`/`channels`/`filter_config`` tables and their ``db.load_filter_config`` / ``upsert_channel`` / ``insert_video`` repos stay working as shims over the new tables (same pattern as the Track-A ``User`` shim), so existing call sites in ``telethon_client.py``, ``backfill.py``, and ``commands.py`` keep working without a flag day.

Consequences

Positive

- Provider-neutral schema: ``sources``, ``content_items``, ``policies``, ``destinations``, ``routing_rules``, ``delivery_attempts`` all carry a ``provider`` column or provider-prefixed keys. Adding WhatsApp later is a new ``provider`='whatsapp'` value, not a new schema.
- ``dedup_key`` (Option 1A) generalizes ADR 0006 without losing any dedup quality and without downloading anything.
- ``Policy`` (Option 2A) gives a single home for filter + future enrichment config, enabling ADR 0009's per-route overrides with no second migration.
- Backwards-compatible via shims, mirroring the Track-A pattern that already passed review.

Negative

- Six new tables; the schema grows significantly. Each is small and single-purpose, but the surface area is larger.
- The dedup-key derivation logic is in code, not the schema ? needs its own unit tests (a ``compute_dedup_key(provider, **metadata)`` pure function).
- Phase 1 carries some dead-on-arrival columns (``content_items.link_url``, ``text``) that aren't populated until later tracks. This is intentional (avoid future migrations) but means some columns are NULL until then.

Alternatives considered

- **Do nothing; keep ``channels`/`videos`/`filter_config`` and special-case providers in code.** Rejected ? same conflation argument as ADR 0010. Telegram-specific ids in column names (``channel_id``, ``message_id``, ``document_id``) would leak into every future provider's code path.
- **Land all six tables' *logic* in this track.** Rejected ? Track B is already the largest track. Splitting schema (Track B) from delivery logic (Track C) keeps each PR reviewable. The empty ``destinations`` / ``routing_rules`` / ``delivery_attempts`` tables in this track are the contract Track C codes against.
- **``content_items`` as a view over ``videos`` (no migration).** Rejected ? views don't give us the ``provider`` column or the ``dedup_key`` index, and SQLite view support for writes is limited. A real table is needed.

Open questions for the decision owner

1. **Question 1** ? ``dedup_key``: 1A (tiered, recommended), 1B (hash, rejected), or 1C (provider-assigned only).
2. **Question 2** ? ``Policy`` ? ``filter_config``: 2A (generalize, recommended) or 2B (wrap, rejected).
3. **Dead columns in Phase 1** ? acceptable to land ``content_items.link_url`` / ``text`` as NULL until later tracks populate them? Recommendation: yes

- (avoids a future ``ALTER TABLE``).
4. ****Deprecated-table retention**** ? one release (default), matching ADR 0010?
Recommendation: yes.

References

- ADR 0003 ? metadata-first probing (the no-download constraint that rules out Option 1B's content hashing).
- ADR 0005 ? per-user delivery target (generalized by ``destinations`` / ``routing_rules``).
- ADR 0006 ? content dedup (generalized by ``content_items.dedup_key``, Option 1A).
- ADR 0009 ? selective scanning + multiplexed routing (depends on ``sources.watched`` and ``routing_rules``, both defined here).
- ADR 0010 ? identity/session separation (this ADR's ``account_id`` FKs point at ``accounts.id``).
- Issue #27 ? Track B (this ADR's tracking issue).
- Issue #25 ? umbrella; defines the six concepts and the implementation order.

7. assistant (2026-07-06T19:16:14.670Z)

Now let me check the live handler and how ``text`` gets populated, plus ADR 0003 and the delivery/dedup code.

8. user (2026-07-06T19:16:16.581Z)

"""Telethon ingestion: client factory, event handler, and metadata extraction.

Architecture (see docs/design.md §5):

```
NewMessage event
-> extract_video_meta(message)    # pure, unit-tested
-> dedup via db.get_video()
-> FilterEngine.evaluate(meta)
-> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
```

The metadata extraction is deliberately a ****pure**** function taking a Telethon ``Message`` and returning a ``VideoMeta | None``. This isolates the only piece of logic that's worth unit-testing (everything else is Telethon/async glue).

"""

```
from __future__ import annotations
```

```
import logging
import sqlite3
from pathlib import Path
from typing import Protocol
```

```
import aiosqlite
from telethon import TelegramClient
from telethon.events import NewMessage
from telethon.tl.types import (
    Document,
    DocumentAttributeAnimated,
    DocumentAttributeVideo,
    Message,
    MessageMediaDocument,
)
```

```
from tg_video_filter import db
from tg_video_filter.commands import ENGINE_KEY
from tg_video_filter.delivery import build_link_text, deliver
from tg_video_filter.filters import FilterEngine, VideoMeta
from tg_video_filter.models import Channel, DeliveryMode, TelegramSession, User, VideoRecord
```

```
log = logging.getLogger(__name__)
```

--- pure metadata extraction -----

```
def extract_video_meta(message: Message) -> VideoMeta | None:
    """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.

    Returns ``None`` when the message has no video/document media. Returns a
    ``VideoMeta`` (with fields possibly ``None``) for a video whose individual
    attributes are missing ? the filter engine decides how to treat gaps.

    Telegram stores video metadata in two places:
    - ``Document``: ``size`` (bytes), ``mime_type`` (str), ``attributes`` (list)
    - ``DocumentAttributeVideo`` (inside ``attributes``): ``duration`` (float s),
      ``w`` (width), ``h`` (height)

    Note: we inspect ``message.media`` directly rather than the ``.video``
    convenience property, because the latter only exists on Telethon's *custom*
    Message wrapper (applied at runtime) and is ``None`` on the raw TL Message
    used in tests. A Document is considered a video iff it contains a
    ``DocumentAttributeVideo``.
    """
    media = getattr(message, "media", None)

    # Real Telegram messages wrap documents in MessageMediaDocument.
    # Unwrap it to get the actual Document.
    if isinstance(media, MessageMediaDocument):
        media = media.document

    if not isinstance(media, Document):
        return None

    # Only treat this as a video if the document carries a video attribute.
    has_video_attr = any(isinstance(a, DocumentAttributeVideo) for a in media.attributes)
    if not has_video_attr:
        return None

    meta = VideoMeta(
        size_bytes=media.size,
        mime_type=media.mime_type or None,
    )

    # Find the video attribute for duration/dimensions and round/nosound flags.
    video_update: dict = {}
    for attr in media.attributes:
        if isinstance(attr, DocumentAttributeVideo):
            if attr.duration:
                video_update["duration_s"] = int(round(attr.duration))
            if attr.w:
                video_update["width"] = attr.w
            if attr.h:
                video_update["height"] = attr.h
            video_update["is_round"] = attr.round_message # may be None
            video_update["nosound"] = attr.nosound # may be None
            break

    # Check for GIF (DocumentAttributeAnimated present).
    is_gif = any(isinstance(a, DocumentAttributeAnimated) for a in media.attributes)

    meta = meta.model_copy(update={**video_update, "is_gif": is_gif})
    return meta
```

--- client factory -----

```
def build_client(
    session: TelegramSession, settings: SettingsLike
) -> TelegramClient: # pragma: no cover
    """Build a Telethon client with a file-backed SQLiteSession (ADR 0010).

    The session path is read from the ``telegram_sessions.session_path`` column
    (data-driven, no longer derived from the PK at runtime) and resolved
    against ``settings.sessions_dir``. The directory is created if missing.
    """
    sessions_dir = Path(settings.sessions_dir)
    sessions_dir.mkdir(parents=True, exist_ok=True)
    # session_path is stored as a basename (e.g. "user_111.session"); join with
    # the runtime sessions_dir so the migration did not need to know the path.
    session_path = sessions_dir / session.session_path

    client = TelegramClient(
        str(session_path),
        api_id=settings.tg_api_id,
        api_hash=settings.tg_api_hash,
    )
    return client


class SettingsLike(Protocol):
    """Structural type for the subset of Settings the client factory needs.

    Using a Protocol keeps the module importable in tests without dragging in
    the real Settings class. At runtime ``config.Settings`` satisfies this shape.
    """

    tg_api_id: int
    tg_api_hash: str
    sessions_dir: str
```

--- event handler wiring -----

```
def make_new_message_handler(
    *,
    user: User,
    conn: aiosqlite.Connection,
    engine_holder: dict[str, FilterEngine],
    client: TelegramClient,
    delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
    delivery_target: str = "me",
) -> _NewMessageCallback:
    """Return an async callback for ``events.NewMessage``.

    The callback:
    1. extracts VideoMeta + document_id (skips if not a video),
    2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
       plus the existing (channel, message) instance dedup,
    3. runs the filter engine (read from ``engine_holder`` so config
       changes via ``/set`` take effect immediately),
    4. inserts a VideoRecord row,
    5. delivers a match to ``delivery_target`` (see ADR 0005).
    """

    async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
        message: Message = event.message # type: ignore[assignment]
```

```

if message is None:
    return # Telethon edge case: event has no message (shouldn't happen)
# Only process genuine channel/group posts and private messages we
# receive ? outgoing messages (sent by our own account) are ignored.
if getattr(message, "out", False):
    return

meta = extract_video_meta(message)
if meta is None:
    return # not a video

channel_id = _peer_id(message.peer_id)
message_id = message.id
document_id = _extract_document_id(message)

# --- watched gate (ADR 0009, Track E) -----
# Opt-in: a source is ingested only when watched=True. An absent
# sources row means the channel has never been seen ? treat it as
# unwatched and lazily create a watched=0 row for /channels
# discoverability (ADR 0009 lines 37-44, 130-131).
if not await db.should_ingest_source(conn, user.id, channel_id):
    return # skip unwatched / unknown source

# --- three-tier dedup (cheapest first) ---

# Tier 0: same (user, channel, message) instance ? already processed.
existing = await db.get_video(conn, user.id, channel_id, message_id)
if existing is not None:
    return

# Tier 1: same Telegram document.id ? forwarded/cross-posted file.
if document_id is not None:
    existing = await db.get_video_by_document_id(conn, user.id, document_id)
    if existing is not None:
        log.info(
            "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
            "original_channel=%s original_msg=%s",
            user.id,
            document_id,
            channel_id,
            message_id,
            existing.channel_id,
            existing.message_id,
        )
        return

# Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
if (
    meta.size_bytes is not None
    and meta.duration_s is not None
    and meta.height is not None
    and meta.width is not None
):
    existing = await db.get_video_by_content_fingerprint(
        conn,
        user.id,
        meta.size_bytes,
        meta.duration_s,
        meta.height,
        meta.width,
    )
    if existing is not None:
        log.info(
            "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
            "channel=%s msg=%s original_channel=%s original_msg=%s",

```

```

        user.id,
        meta.size_bytes,
        meta.duration_s,
        meta.height,
        meta.width,
        channel_id,
        message_id,
        existing.channel_id,
        existing.message_id,
    )
    return

# Lazily record the channel (best-effort; failures are non-fatal).
try:
    await db.upsert_channel(
        conn,
        Channel(user_id=user.id, channel_id=channel_id, title=_channel_title(event)),
    )
except Exception:
    log.warning(
        "failed to upsert channel %s for user %s", channel_id, user.id, exc_info=True
    )

result = engine_holder[ENGINE_KEY].evaluate(meta)
record = VideoRecord(
    user_id=user.id,
    channel_id=channel_id,
    message_id=message_id,
    document_id=document_id,
    duration_s=meta.duration_s,
    width=meta.width,
    height=meta.height,
    size_bytes=meta.size_bytes,
    mime_type=meta.mime_type,
    matched=result.matched,
)
try:
    inserted = await db.insert_video(conn, record)
except sqlite3.IntegrityError:
    # Lost a dedup race: a concurrent delivery of the same file
    # (same document_id or same message instance) won the insert.
    # The three-tier check above already classified this as new, so
    # the collision means another task is processing (or has
    # processed) it ? log and skip.
    log.info(
        "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
        user.id,
        channel_id,
        message_id,
        document_id,
    )
    return
log.info(
    "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
    user.id,
    channel_id,
    message_id,
    result.matched,
    result.reasons,
)

if result.matched:
    # Build link text for LINK/BOTH modes (None for FORWARD).
    link_text = None
    if delivery_mode != DeliveryMode.FORWARD:

```



```

        channel_title = _channel_title(event)
        link_text = build_link_text(
            message_link=getattr(message, "link", None),
            duration_s=meta.duration_s,
            height=meta.height,
            size_bytes=meta.size_bytes,
            channel_title=channel_title,
        )
    await deliver(
        client,
        message,
        conn,
        inserted.id,
        delivery_mode,
        link_text=link_text,
        target=delivery_target,
    )

return on_new_message

```

--- helpers -----

```

def _extract_document_id(message: Message) -> int | None:
    """Extract Telegram's ``document.id`` from a video message.

    Returns None if the message has no document media or the id is absent.
    This is the most reliable content-based dedup key: Telegram reuses the
    same ``document.id`` when a file is forwarded across channels.
    """
    media = getattr(message, "media", None)
    if isinstance(media, MessageMediaDocument):
        media = media.document
    if isinstance(media, Document):
        return getattr(media, "id", None)
    return None

def _peer_id(peer: object) -> int:
    """Best-effort numeric id for the peer of a message.

    Telegram represents channel/group/user peers as different TL types; we
    collapse them to a single int for dedup/storage. The exact subclass isn't
    important ? only that the id is stable per chat.
    """
    # Each Peer* type has a field like channel_id / chat_id / user_id.
    for field in ("channel_id", "chat_id", "user_id"):
        val = getattr(peer, field, None)
        if val is not None:
            return int(val)
    # Fallback: hash the repr so we never collide with real ids (negative).
    return -abs(hash(repr(peer)) % (2**31))

def _channel_title(event: NewMessage.Event) -> str | None: # pragma: no cover
    """Best-effort channel title from the event, if available."""
    chat = getattr(event, "chat", None)
    return getattr(chat, "title", None) if chat is not None else None

class _NewMessageCallback:
    """Structural type annotation for the handler returned by make_*_handler."""

```

```

    async def __call__(self, event: NewMessage.Event) -> None: ...

__all__ = [
    "build_client",
    "extract_video_meta",
    "make_new_message_handler",
]

```

9. assistant (2026-07-06T19:16:21.334Z)

The handler uses the legacy `VideoRecord`/`videos` shim path, not `ContentItem`. And `text` (caption) is never populated. Let me confirm the `text` claim and check ADR 0003 and whether `ContentItem` carries caption. Also check whether `extract\_video\_meta` or the handler reads `message.message` (the caption text).

10. user (2026-07-06T19:16:23.627Z)

```

main:src/tg_video_filter/db.py:5:for the provider-neutral ``sources`` / ``content_items`` /
``policies`` /
main:src/tg_video_filter/db.py:22:     ContentItem,
main:src/tg_video_filter/db.py:87:-- ``content_items``, and ``policies`` are populated by
migration 003 (and by
main:src/tg_video_filter/db.py:105:CREATE TABLE IF NOT EXISTS content_items (
main:src/tg_video_filter/db.py:169:     content_item_id INTEGER NOT NULL REFERENCES
content_items(id),
main:src/tg_video_filter/db.py:178:     UNIQUE (content_item_id, destination_id)
main:src/tg_video_filter/db.py:255:# Pure function: computes the ``content_items.dedup_key``
value from metadata
main:src/tg_video_filter/db.py:273:     """Compute the provider-neutral ``dedup_key`` for a
``ContentItem``.
main:src/tg_video_filter/db.py:508:     Creates ``sources``, ``content_items``, ``policies``,
``destinations``,
main:src/tg_video_filter/db.py:551:     # 2. Backfill content_items from videos in two phases
(review findings #2 & #3):
main:src/tg_video_filter/db.py:559:         INSERT INTO content_items
main:src/tg_video_filter/db.py:585:         UPDATE content_items SET dedup_key = 'tg:doc:' ||
document_id
main:src/tg_video_filter/db.py:589:         SELECT MIN(id) FROM content_items
main:src/tg_video_filter/db.py:599:         UPDATE content_items SET dedup_key =
SELECT MIN(id) FROM content_items
main:src/tg_video_filter/db.py:607:         # produce a content_items row. A mismatch signals
fingerprint collisions
main:src/tg_video_filter/db.py:646:     async with conn.execute("SELECT COUNT(*) FROM
content_items") as cur:
main:src/tg_video_filter/db.py:653:         "migration_003: %d videos rows produced %d
content_items rows "
main:src/tg_video_filter/db.py:741:         (content_item_id, destination_id, route_id,
status, attempt_count,
main:src/tg_video_filter/db.py:746:         JOIN content_items ci
main:src/tg_video_filter/db.py:793:         f"types are Telegram-specific; use
ContentItem/Source directly for "
main:src/tg_video_filter/db.py:857:def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
main:src/tg_video_filter/db.py:858:     return ContentItem(
main:src/tg_video_filter/db.py:921:         content_item_id=row["content_item_id"],
main:src/tg_video_filter/db.py:1165:# --- ContentItem repos (ADR 0011)
-----
main:src/tg_video_filter/db.py:1168:async def insert_content_item(conn: aiosqlite.Connection,
item: ContentItem) -> ContentItem:
main:src/tg_video_filter/db.py:1177:     """INSERT INTO content_items
main:src/tg_video_filter/db.py:1188:         item.text,
main:src/tg_video_filter/db.py:1206:async def get_content_item_by_message(
main:src/tg_video_filter/db.py:1211:) -> ContentItem | None:
main:src/tg_video_filter/db.py:1215:     """SELECT * FROM content_items
main:src/tg_video_filter/db.py:1220:     return _row_to_content_item(row) if row else None

```

```

main:src/tg_video_filter/db.py:1223:async def get_content_item_by_dedup_key(
main:src/tg_video_filter/db.py:1225:) -> ContentItem | None:
main:src/tg_video_filter/db.py:1234:         "SELECT * FROM content_items WHERE account_id = ?
AND dedup_key = ?",
main:src/tg_video_filter/db.py:1238:         return _row_to_content_item(row) if row else None
main:src/tg_video_filter/db.py:1342:         content_item_id: int,
main:src/tg_video_filter/db.py:1348:         """Record a delivery outcome for one (content_item,
destination) pair.
main:src/tg_video_filter/db.py:1357:         (content_item_id, destination_id, route_id,
status, attempt_count,
main:src/tg_video_filter/db.py:1361:         ON CONFLICT(content_item_id, destination_id) DO
UPDATE SET
main:src/tg_video_filter/db.py:1368:         content_item_id,
main:src/tg_video_filter/db.py:1381:         conn: aiosqlite.Connection, content_item_id: int,
destination_id: int
main:src/tg_video_filter/db.py:1386:         "SELECT * FROM delivery_attempts WHERE
content_item_id = ? AND destination_id = ?",
main:src/tg_video_filter/db.py:1387:         (content_item_id, destination_id),
main:src/tg_video_filter/db.py:1446:# --- legacy VideoRecord shims (backwards-compatible view
over content_items) -
main:src/tg_video_filter/db.py:1452:         """Deprecated shim: Tier-0 lookup against
`content_items` (ADR 0011).
main:src/tg_video_filter/db.py:1455:         :class:`Source` + :class:`ContentItem` and projects the
latter onto a
main:src/tg_video_filter/db.py:1456:         :class:`VideoRecord`. New code should call
:func:`get_content_item_by_message`.
main:src/tg_video_filter/db.py:1461:         item = await get_content_item_by_message(conn, user_id,
src.id, str(message_id))
main:src/tg_video_filter/db.py:1462:         return _content_item_to_video(item, src, user_id,
channel_id) if item else None

```

11. user (2026-07-06T19:16:25.742Z)

ADR 0003: Metadata-first probing (no download in the hot path)

- **Status**: Accepted
- **Date**: 2026-07-03

Context

To filter videos we need properties: duration, resolution (height), file size, MIME type, and (later) codec/bitrate. There are two ways to obtain them:

1. **Read Telegram message metadata** ? every video `Message` carries a `Document` whose attributes include `DocumentAttributeVideo` (duration, `w`, `h`) plus `document.size` and `document.mime_type`. This is **free** ? no network transfer, no disk.
2. **Download the file and run `ffprobe`** ? gives the **true** codec, bitrate, and exact stream dimensions, at the cost of downloading (potentially gigabytes per video), needing `ffmpeg` installed, and adding latency to the hot path.

For the v1 filters (duration, height ? target, size ? max), the metadata fields are exactly what we need. Codec/bitrate accuracy is only relevant for future codec-level filters, which are explicitly out of v1 scope.

Decision

In the ingestion hot path, build `VideoMeta` **only** from Telegram message metadata (no download). True probing via `ffprobe` is available only through an explicit, on-demand `/inspect` command ? itself deferred to v2.

Consequences

****Positive****

- Ingestion is cheap and fast: no bandwidth, no disk, no `ffmpeg` dependency.
- Latency from "video posted" to "forwarded to Saved Messages" is minimal.
- Container image stays small (no ffmpeg).

****Negative****

- "1080p" is inferred as `height == 1080`. Telegram's metadata is usually accurate for dimension, but if a sender mislabels a re-encode, we trust the label. Acceptable for v1's quality-bar use case.
- Codec-level rules (e.g. "only H.264, skip AV1") are impossible without downloading ? hence their deferral to v2 alongside `/inspect`.

Alternatives considered

- ****Always download + ffprobe****: rejected for the hot path due to bandwidth, storage, latency, and the added `ffmpeg` runtime dependency. Could be added as a per-channel opt-in for power users in a future version.
- ****Metadata first, ffprobe on demand**** (the v2 plan): keep the hot path cheap and offer an explicit deep-inspect command. This is the chosen evolution path.

-----DB record_delivery-----

```
main:src/tg_video_filter/db.py:6:``destinations`` / ``routing_rules`` / ``delivery_attempts``
schema (ADR 0011).
main:src/tg_video_filter/db.py:88:-- the shims below); ``destinations`` / ``routing_rules`` /
``delivery_attempts``
main:src/tg_video_filter/db.py:167:CREATE TABLE IF NOT EXISTS delivery_attempts (
main:src/tg_video_filter/db.py:174:     last_error      TEXT,
main:src/tg_video_filter/db.py:509:     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT
EXISTS`` ? on fresh
main:src/tg_video_filter/db.py:674:     Also backfills ``delivery_attempts`` rows from the legacy
``videos_deprecated``
main:src/tg_video_filter/db.py:737:     # 3. Backfill delivery_attempts from previously-forwarded
videos.
main:src/tg_video_filter/db.py:740:         INSERT OR IGNORE INTO delivery_attempts
main:src/tg_video_filter/db.py:926:         last_error=row["last_error"],
main:src/tg_video_filter/db.py:1339:async def record_delivery(
main:src/tg_video_filter/db.py:1356:         """INSERT INTO delivery_attempts
main:src/tg_video_filter/db.py:1358:         last_error, created_at, delivered_at)
main:src/tg_video_filter/db.py:1363:         attempt_count=delivery_attempts.attempt_count +
1,
main:src/tg_video_filter/db.py:1364:         last_error=excluded.last_error,
main:src/tg_video_filter/db.py:1366:         ELSE
delivery_attempts.delivered_at END""",
main:src/tg_video_filter/db.py:1386:         "SELECT * FROM delivery_attempts WHERE
content_item_id = ? AND destination_id = ?",
main:src/tg_video_filter/db.py:1572:async def should_ingest_source(
main:src/tg_video_filter/db.py:1652:     Per-destination delivery state moves to
``delivery_attempts`` (Track C,
main:src/tg_video_filter/db.py:1795:     "record_delivery",
```

12. assistant (2026-07-06T19:16:29.983Z)

Let me confirm two things: (1) `insert\_content\_item` writes `item.text` but no caller populates text from a message; (2) `record\_delivery` signature and whether short-circuited items get a row. Let me look at record_delivery and the FilterEngine, and confirm whether anything sets ContentItem.text.

13. user (2026-07-06T19:16:31.013Z)

```
1339     async def record_delivery(
1340         conn: aiosqlite.Connection,
1341         *,
1342         content_item_id: int,
1343         destination_id: int,
```

```

1344     route_id: int | None = None,
1345     success: bool,
1346     error: str | None = None,
1347 ) -> None:
1348     """Record a delivery outcome for one (content_item, destination) pair.
1349
1350     This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
1351     sets status=delivered; on failure, increments attempt_count and records the
1352     error. Idempotent: repeated calls update the same row.
1353     """
1354     status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
1355     await conn.execute(
1356         """INSERT INTO delivery_attempts
1357             (content_item_id, destination_id, route_id, status, attempt_count,
1358              last_error, created_at, delivered_at)
1359             VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
1360                     CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
1361             ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
1362                 status=excluded.status,
1363                 attempt_count=delivery_attempts.attempt_count + 1,
1364                 last_error=excluded.last_error,
1365                 delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
1366                             ELSE delivery_attempts.delivered_at END""",
1367         (
1368             content_item_id,
1369             destination_id,
1370             route_id,
1371             status.value,
1372             error,
1373             success,
1374             success,
1375         ),
1376     )
1377     await conn.commit()
1378
1379
1380     async def get_delivery_attempt(
1381         conn: aiomysql.Connection, content_item_id: int, destination_id: int
1382     ) -> DeliveryAttempt | None:
1383         """Read a delivery attempt row by its natural key (idempotency boundary)."""
1384         conn.row_factory = aiomysql.Row
1385         async with conn.execute(
1386             "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id = ?"
1387             (content_item_id, destination_id),
1388         ) as cur:

```

14. user (2026-07-06T19:16:34.529Z)

```

model_config extra: {}
dumped: {'min_duration_s': 5, 'max_duration_s': 7200, 'target_height': 1080, 'max_size_mb':
2048, 'exclude_gif': True, 'exclude_round': True, 'exclude_nosound': False, 'delivery_mode':
<DeliveryMode.FORWARD: 'forward'>, 'delivery_target': 'me'}
has_topics attr: False

```

15. assistant (2026-07-06T19:16:40.501Z)

Key finding confirmed: `record\_delivery` requires `content\_item\_id` AND `destination\_id` (FK to a delivered content item), created only after insert+delivery. And `delivery\_attempts.last\_error` is scoped by `(content\_item\_id, destination\_id)` ? a rejected item that short-circuits before delivery has no destination and no delivery_attempts row. Also confirmed: `FilterConfig.model\_config = {}` (no `extra="forbid"`) so extra keys are ****dropped silently**** (default Pydantic v2 `ignore`), not preserved ? so the `topics`/`allowlist`/`deep\_inspect` keys don't survive a round-trip through `FilterConfig`.

Let me verify the round-trip drop concretely (save then load), and confirm nothing populates `content\_item.text`.

16. user (2026-07-06T19:16:45.689Z)

```
persisted blob: {"min_duration_s":5,"max_duration_s":7200,"target_height":1080,"max_size_mb":2048,"exclude_gif":true,"exclude_round":true,"exclude_nosound":false,"delivery_mode":"forward","delivery_target":"me"}
topics survived round-trip: False
----- does anything set ContentItem.text? -----
```

17. assistant (2026-07-06T19:16:48.802Z)

Both confirmed. Let me check how `save\_filter\_config`/`load\_filter\_config` actually serialize (to be precise about the drop being real in the shim path) and how `insert\_content\_item` is called (is text ever passed).

18. user (2026-07-06T19:16:51.355Z)

```
main:src/tg_video_filter/db.py:83: config_json TEXT NOT NULL
main:src/tg_video_filter/db.py:132: config_json TEXT NOT NULL,
main:src/tg_video_filter/db.py:476: config_json TEXT NOT NULL
main:src/tg_video_filter/db.py:479: "user_id, config_json",
main:src/tg_video_filter/db.py:622: INSERT OR IGNORE INTO policies (account_id, name,
config_json, is_default,
main:src/tg_video_filter/db.py:624: SELECT user_id, 'default', config_json, 1,
main:src/tg_video_filter/db.py:695: CASE WHEN json_valid(config_json)
main:src/tg_video_filter/db.py:696: THEN json_extract(config_json,
'$.delivery_target') END,
main:src/tg_video_filter/db.py:701: CASE WHEN json_valid(config_json)
main:src/tg_video_filter/db.py:702: THEN json_extract(config_json,
'$.delivery_target') END,
----insert_content_item callers----
main:src/tg_video_filter/db.py:1168:async def insert_content_item(conn: aiosqlite.Connection,
item: ContentItem) -> ContentItem:
main:src/tg_video_filter/db.py:1645: item = await insert_content_item(conn, item)
main:tests/test_db.py:1069: item = await db.insert_content_item(
main:tests/test_db.py:1103: await db.insert_content_item(
main:tests/test_db.py:1115: await db.insert_content_item(
main:tests/test_db.py:1295: await db.insert_content_item(
main:tests/test_db.py:1326: await db.insert_content_item(
main:tests/test_db.py:1547: item = await db.insert_content_item(
main:tests/test_db.py:1576: item = await db.insert_content_item(
main:tests/test_db.py:1610: item = await db.insert_content_item(
```

19. user (2026-07-06T19:16:56.298Z)

```
main:src/tg_video_filter/db.py:1711:async def save_filter_config(conn: aiosqlite.Connection,
user_id: int, cfg: FilterConfig) -> None:
    kind="channel",
    watched=False,
),
)
assert src.id is not None # upsert_source populates the surrogate PK
item = ContentItem(
    account_id=video.user_id,
    source_id=src.id,
    provider="telegram",
    provider_message_id=str(video.message_id),
    content_type="video",
    document_id=str(video.document_id) if video.document_id is not None else None,
    duration_s=video.duration_s,
    width=video.width,
    height=video.height,
```

```

        size_bytes=video.size_bytes,
        mime_type=video.mime_type,
        dedup_key=compute_dedup_key(
            provider="telegram",
            document_id=(str(video.document_id) if video.document_id is not None else None),
            source_id=src.id,
            provider_message_id=str(video.message_id),
            size_bytes=video.size_bytes,
            duration_s=video.duration_s,
            height=video.height,
            width=video.width,
        ),
        matched=video.matched,
        seen_at=video.seen_at,
    )
    item = await insert_content_item(conn, item)
    return _content_item_to_video(item, src, video.user_id, video.channel_id)

```

```

async def mark_forwarded(conn: aiomysqlite.Connection, video_id: int) -> None:
    """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).

```

```

    Per-destination delivery state moves to ``delivery_attempts`` (Track C,
    #28). For Phase 1 (before Track C lands) there is no equivalent column on
    ``content_items`` ? the ``matched`` flag alone is sufficient because v1
    delivery still targets the single ``delivery_target`` and retried delivery
    is harmless (forwards are idempotent). This shim is retained so existing
    call sites in ``delivery.py`` keep working without a flag day.
    """

```

```

    # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
    # The row's ``matched`` flag (set at insert time) is the surviving signal.

```

20. assistant (2026-07-06T19:16:59.856Z)

Confirmed: the `insert\_video` shim constructs `ContentItem` without `text=` (it's omitted ? NULL), and the live handler goes through this legacy `VideoRecord`/`insert\_video` shim path, never passing caption text. So `ContentItem.text` is NULL in the live path.

Let me check `save\_filter\_config` to nail down the round-trip drop in the real persistence path.

21. user (2026-07-06T19:17:00.820Z)

```

1711 async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1712     """Deprecated shim: upsert the account's default :class:`Policy`."""
1713     existing = await get_default_policy(conn, user_id)
1714     await upsert_policy(
1715         conn,
1716         Policy(
1717             id=existing.id if existing is not None else None,
1718             account_id=user_id,
1719             name="default",
1720             config_json=cfg.model_dump_json(),
1721             is_default=True,
1722             created_at=(existing.created_at if existing is not None else _now()),
1723             updated_at=_now(),
1724         ),
1725     )
1726
1727
1728 # --- bulk helpers (used by the manager on startup) -----
1729
1730
1731 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1732     conn: aiomysqlite.Connection, user_id: int, limit: int = 100

```

```

1733 ) -> list[VideoRecord]:
1734     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1735     conn.row_factory = aiosqlite.Row
1736     async with conn.execute(
1737         """SELECT ci.*, s.provider_source_id AS src_pid
1738            FROM content_items ci
1739            JOIN sources s ON s.id = ci.source_id
1740            WHERE ci.account_id = ?
1741            ORDER BY ci.id DESC LIMIT ?""",
1742         (user_id, limit),
1743     ) as cur:
1744         rows = await cur.fetchall()
1745     return [
1746         VideoRecord(
1747             id=r["id"],
1748             user_id=user_id,
1749             channel_id=_safe_int_cast(r["src_pid"], "provider_source_id"),
1750             message_id=_safe_int_cast(r["provider_message_id"], "provider_message_id"),
1751             document_id=(
1752                 _safe_int_cast(r["document_id"], "document_id")
1753                 if r["document_id"] is not None
1754                 else None
1755             ),

```

22. assistant (2026-07-06T19:17:05.349Z)

Confirmed the critical drop: `save\_filter\_config` persists `cfg.model\_dump\_json()` where `cfg` is a `FilterConfig` ? which has `model\_config = {}` (default `extra="ignore"`), so any `topics`/`allowlist`/`security\_checks`/`deep\_inspect` keys are dropped on the way in and never round-trip. The ADR's central "backwards-compatible: extra fields are ignored" claim is exactly backwards ? "ignored" means **discarded**, not preserved.

I have enough to report. Let me compile findings.

23. user (2026-07-06T19:18:19.540Z)

Structured output provided successfully